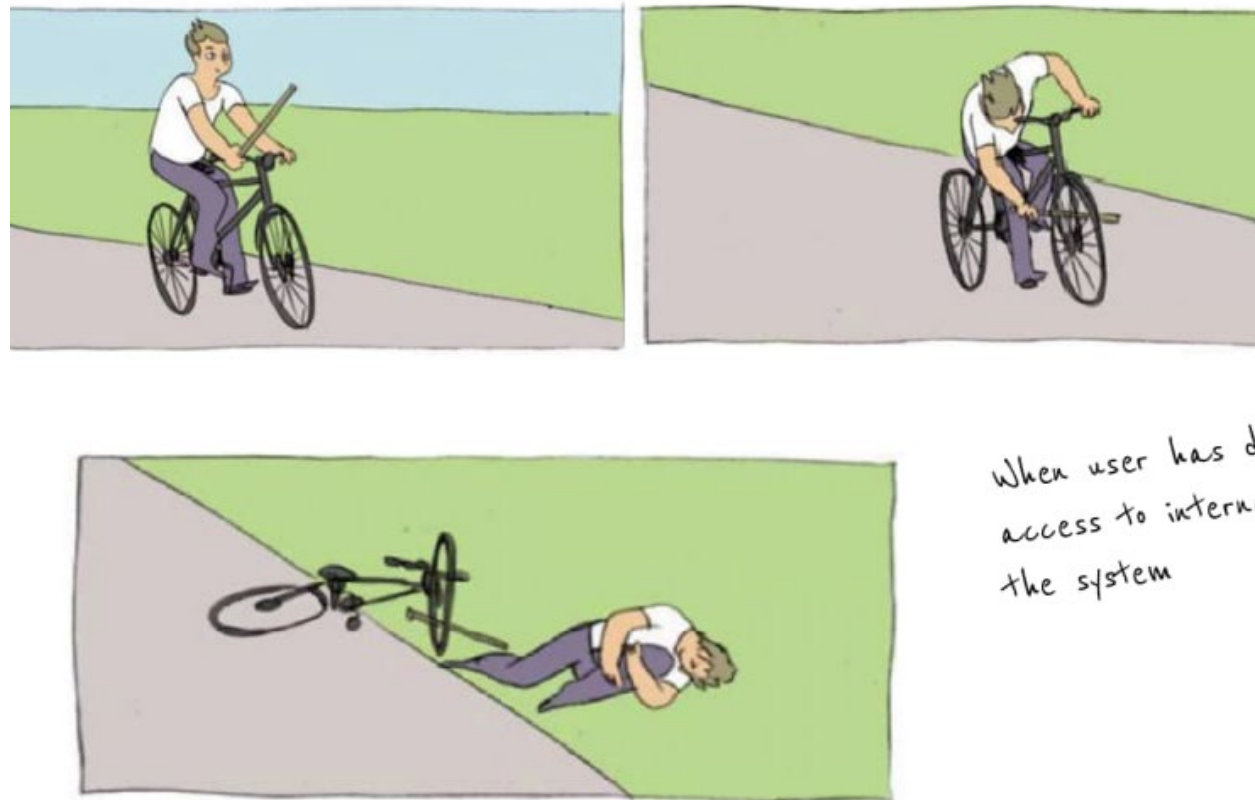


Object Oriented Programming 101

Object Oriented Concepts



- ☐ Encapsulation
- ☐ Inheritance
- ☐ Polymorphism

- ☐ Coupling
- ☐ Cohesion

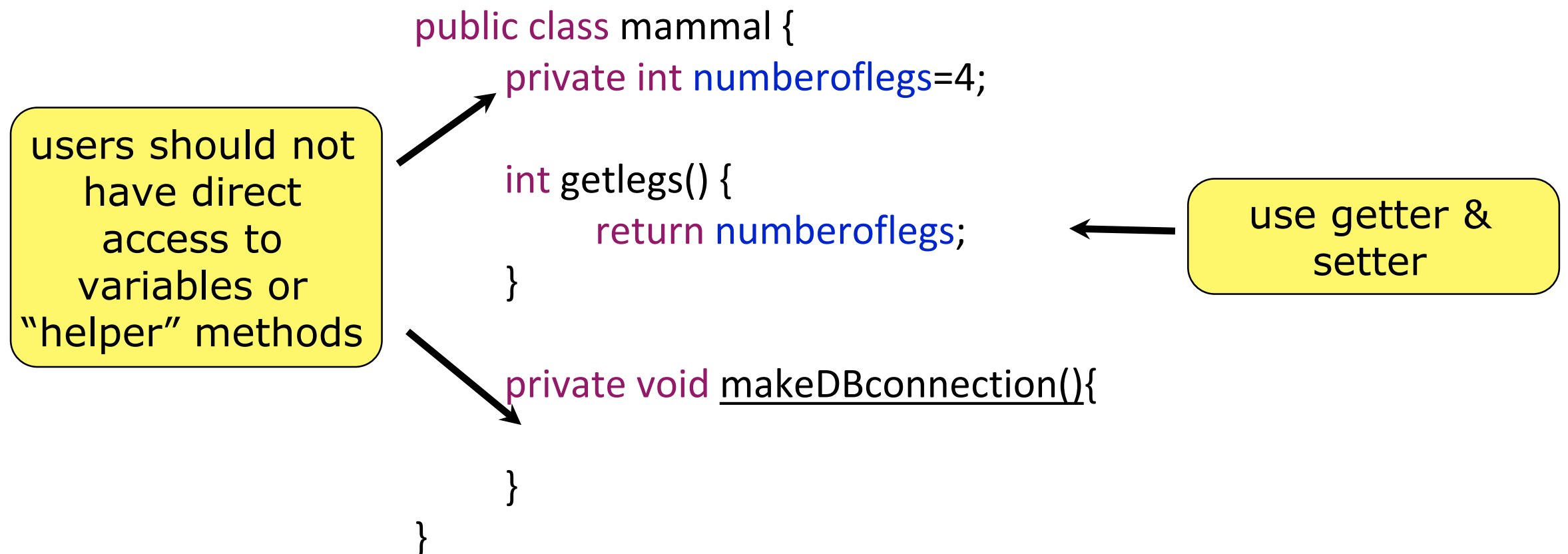
Encapsulation

Encapsulation is the inclusion within a program object of all the resources needed for the object to function - basically, the method and the data. The object is said to "publish its interfaces." Other objects adhere to these interfaces to use the object without having to be concerned with how the object accomplishes it

Protection

Information Hiding

example



Benefits

- maintainability, flexibility and extensibility: modify our implemented code without breaking the code of others who use our code

Inheritance

Inheritance is the concept that when a class of objects is defined, any subclass that is defined can inherit the definitions of one or more general classes

Example

mammal

```
public class mammal {  
    int numberoflegs;  
    int makeSound() { }  
}
```

properties
+
methods

use this
arrow



```
public class cat extends mammal { }
```

```
.....  
cat kitty = new cat();  
kitty.numberoflegs=4;  
kitty.makeSound();  
.....
```

inherit from
mammal

Benefits

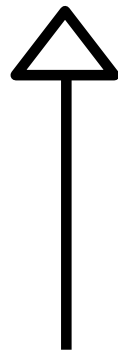
- Object in a subclass need not carry its own definition of data and methods that are generic to the class
 - » **Maintenance**: changes only have to be made in the superclass
 - » **Reliability**: what works for the superclass must work for the subclass.

Polymorphism

Polymorphism (having multiple forms)
- Being able to assign a different meaning to a particular symbol or "operator" in different contexts

Example

mammal



```
mammal catwoman;  
catwoman = new cat();  
catwoman = new human();
```

can be 2
different
objects

```
public class mammal {  
    int numberoflegs=4;  
}
```

```
public class human extends mammal {  
    this.numberoflegs=2;  
    void makeSound() {."hello"... }  
}
```

override
parameter or
method

Benefits

- **Variability**: any object is able to respond differently to some common method or property
- **Maintenance**: objects are independent of each other.

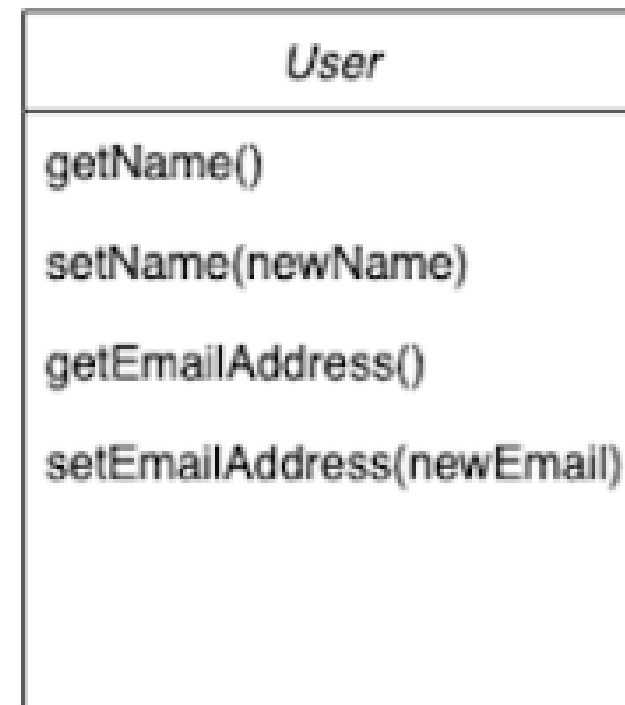
Cohesion

- The degree to which the elements inside a module (or class) belong together
- Higher is better

Example



Low Cohesion



High Cohesion

Benefits

- **Readability**: increases ease of understanding
- **Reusability**: increases use of existing code
- **Complexity**: reduces complexity to a more manageable level

Coupling

- ☐ The degree of interdependency between modules.
- ☐ Tightly coupled modules must change together
- ☐ Looser is better

Example

Tight

If you want to change your skin, you would also have to change the design of your body as well because the two are joined together – they are tightly coupled.

Loose

If you change your shirt, then you are not forced to change your body – when you can do that, then you have loose coupling.

Benefits

- ❑ **Replacing**: increases ease of replacing or swapping code/components
- ❑ **Testability**: easier to test units in isolation
- ❑ **Extendability**: reduces risk of breaking a system when adding new features

Design Patterns

Design Patterns are "descriptions of communicating objects and classes that are customized to solve a general design problem in a particular context." -- Gamma et al.

Benefits

- ☐ Help us solve recurring problems!
- ☐ Provide us with common vocabulary.
- ☐ Improve readability and maintainability.

UML 101

<https://www.visual-paradigm.com/guide/uml-unified-modeling-language/uml-class-diagram-tutorial/>

Java 101

<https://www.tutorialspoint.com/java/index.htm>
<https://www.youtube.com/watch?v=eIrMbAQSU34>

projector

desk

Group 1

Group 2

Group 3

Group 4

Group 5

Group 6

Group 7

Group 8

Group 9

Group 10

Group 11

Group 12

pillar

Group 13

Group 14

Group 15

Group 16

Group 17

Group 18

Group 19

Group 20

Group 21

Group 22

Group 23

Group 24

Group 25

Group 26

Group 27

Group 28

Group 29

pillar

skateboards

door